

Nery Otoniel Colorado Menchú 9490-22-4867

Josué Sebastián Mancilla González 9490-22-1157

Desarrollo Web – Sección B



Guatemala Octubre 2025

Sede el Naranjo Mixco

Table of Contents

Introducción	3
Descripción de Componentes	4
Frontend (React con TypeScript).....	4
Backend (Node.js + Express + MongoDB).....	4
Descripción de Uso de TypeScript.....	5
Frontend (React con TypeScript).....	5
Backend (Node.js + Express con TypeScript)	5
Beneficios Generales	6
Descripción de Uso de SASS/SCSS	6
Diagrama Entidad Relación.....	7



Introducción

El presente manual técnico documenta el diseño, desarrollo e implementación de la plataforma web para la gestión y realización de votaciones electrónicas del Colegio de Ingenieros de Guatemala. Este sistema ha sido desarrollado bajo un stack moderno que incluye React con TypeScript en el frontend, Node.js con Express en el backend, y una base de datos para la persistencia de la información.

El objetivo de este documento es servir como guía de referencia técnica, describiendo la arquitectura del sistema, los componentes utilizados, la estructura de la base de datos, y las decisiones técnicas adoptadas durante el desarrollo. Está dirigido a desarrolladores, administradores de sistema y cualquier persona interesada en comprender el funcionamiento interno de la aplicación.

A lo largo de este manual se detallan los módulos frontend y backend, el uso de SASS/SCSS para los estilos, la aplicación de TypeScript para garantizar la robustez del código, y la integración de JWT para la seguridad de las sesiones, proporcionando así una visión completa del proyecto.

Descripción de Componentes

Frontend (React con TypeScript)

AdminRoute: Componente de ruta protegida que verifica el JWT y rol de administrador antes de permitir acceso a las secciones administrativas.

PrivateRoute: Ruta privada que asegura la autenticación del usuario antes de mostrar contenido restringido a votantes registrados.

Navbar: Barra de navegación responsive que muestra menús diferenciados según el tipo de usuario y permite la navegación entre secciones.

CampaingCard: Componente visual que muestra el resumen de una campaña electoral incluyendo título, descripción, estado y lista de candidatos.

CandidateCard: Tarjeta representativa de un candidato que muestra su información personal y permite la acción de votar cuando corresponde.

VotingChart: Gráfico interactivo que visualiza los resultados de votación en tiempo actualizado usando bibliotecas como Chart.js.

Services (API): Capa centralizada de servicios que maneja todas las comunicaciones HTTP con el backend mediante funciones tipadas.

Backend (Node.js + Express + MongoDB)

Modelos (Esquemas MongoDB):

User: Almacena datos de ingenieros colegiados con campos únicos como colegiadold y dpi.

Campaing: Gestiona las campañas electorales con fechas, estado y límites de votación.

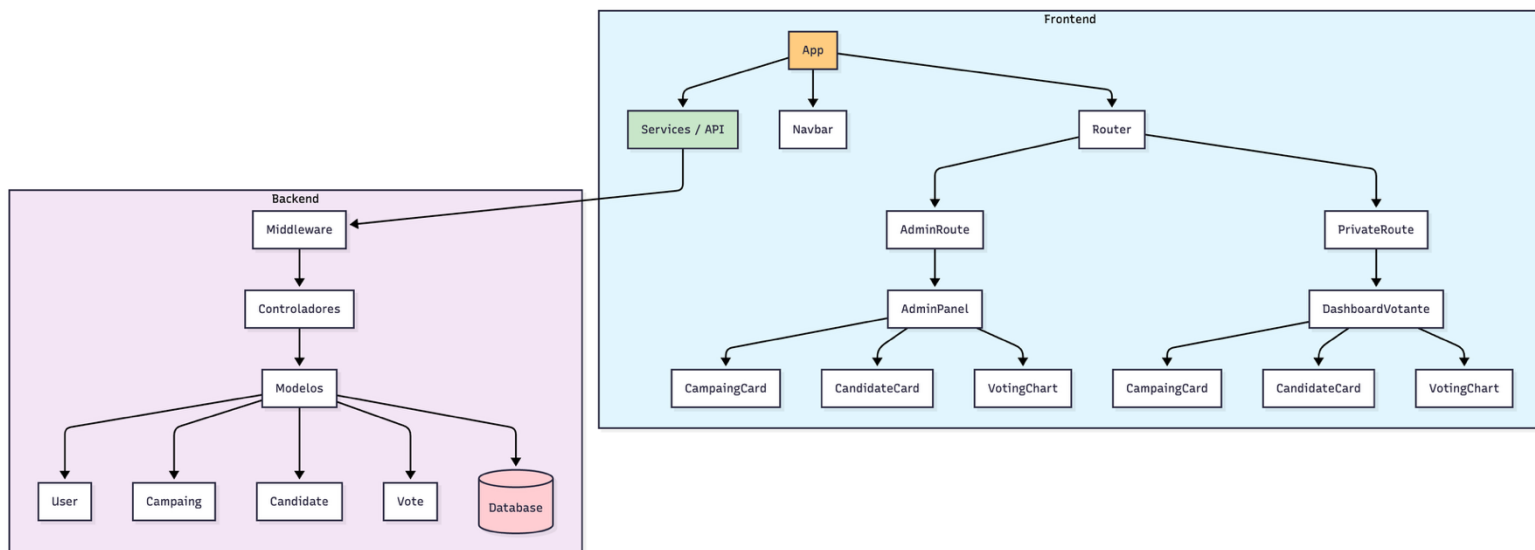
Candidate: Contiene información de candidatos asociados a campañas específicas.

Vote: Registra cada voto emitido con referencias a usuario, candidato y campaña.

Controladores: Módulos que contienen la lógica de negocio para autenticación, gestión de campañas, administración de candidatos y procesamiento de votos.

Middleware: Intermediarios que validan JWT, verifican permisos de administrador y aseguran la integridad de los datos en las peticiones.

Database (MongoDB): Configuración de conexión a la base de datos NoSQL usando Mongoose, con esquemas tipados y relaciones referenciales entre colecciones.



Descripción de Uso de TypeScript

Frontend (React con TypeScript)

TypeScript se implementa para garantizar tipado estático en toda la aplicación. En los componentes React se definen interfaces para las props, estados y eventos, asegurando el uso correcto de datos. Los hooks como `useState` y `useContext` se tipan para prevenir errores en el estado de la aplicación. Los servicios API incluyen interfaces para las respuestas y parámetros, validando la estructura de datos desde el backend.

Backend (Node.js + Express con TypeScript)

En el backend, TypeScript se integra con Mongoose para definir modelos de MongoDB con interfaces tipadas, proporcionando autocompletación y validación en operaciones de base de datos. Los controladores y middlewares utilizan tipos personalizados para objetos `Request` y `Response` de Express, asegurando consistencia en los datos recibidos y enviados. Las rutas definen tipos para parámetros y cuerpos de solicitud, validando estáticamente la API.

Beneficios Generales

El uso de TypeScript en todo el stack proporciona detección temprana de errores, mejor autocompletación en IDEs, código más mantenible y refactoring seguro. Esto garantiza consistencia de datos desde el frontend hasta MongoDB, creando una base sólida para el sistema de votaciones.

Descripción de Uso de SASS/SCSS

Organización Modular

Se utiliza una estructura modular con archivos parciales (`_variables.scss`, `_mixins.scss`) que se importan mediante `@use`. Esto permite separar concerns y mantener el código organizado.

Variables y Mixins Reutilizables

Las variables centralizan colores, fuentes y espaciados para consistencia visual. Los mixins como `@include card` y `@include custom-button` encapsulan estilos repetitivos, evitando duplicación.

Ejemplo uso en mi proyecto:

```
.campaign-card {
  @include card;
  height: 100%;
  cursor: pointer;
  transition: $transition-base;

  &:hover {
    transform: translateY(-5px);
    box-shadow: $box-shadow-lg;
  }
}
```

Anidamiento y Funciones

El anidamiento mejora la legibilidad agrupando estilos relacionados. Las funciones de color como `color.adjust()` permiten variaciones dinámicas de los colores base.

Diagrama Entidad Relación

CAMPAIGNS

Propósito: Gestionar campañas electorales

Datos: fechas, estado, título, descripción

Relación: 1 campaña → Múltiples candidatos y votos

USERS

Propósito: Registrar votantes y administradores

Datos únicos: colegiado, email, DPI

Seguridad: Password hasheada y roles

CANDIDATES

Propósito: Candidatos por campaña

Relación: Pertenece a 1 campaña

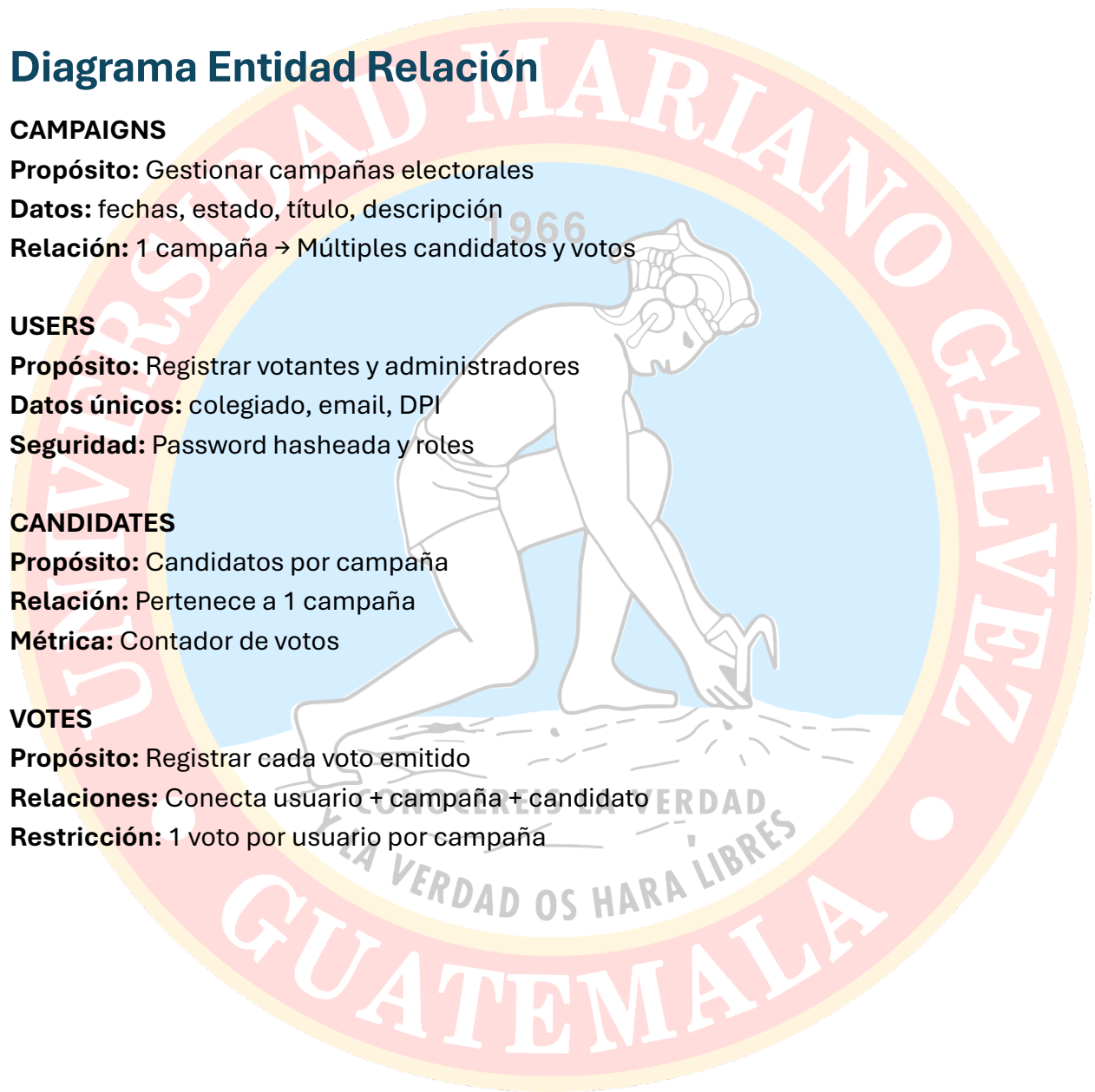
Métrica: Contador de votos

VOTES

Propósito: Registrar cada voto emitido

Relaciones: Conecta usuario + campaña + candidato

Restricción: 1 voto por usuario por campaña



CAMPAIGNS		
ObjectId	_id	Identificador único
String	titulo	Título de la campaña
String	descripcion	Descripción detallada
Date	fechaInicio	Fecha de inicio
Date	fechaFin	Fecha de finalización
Boolean	votacionActiva	Indica si la campaña está activa
Date	createdAt	Fecha de creación
Date	updatedAt	Fecha de actualización

tiene

CANDIDATES		
ObjectId	_id	Identificador único
ObjectId	campaniaId	Referencia a Campaign
String	nombre	Nombre del candidato
String	propuesta	Propuesta del candidato
String	foto	URL de la foto
Number	votos	Contador de votos
Date	createdAt	Fecha de creación
Date	updatedAt	Fecha de actualización

USERS		
ObjectId	_id	Identificador único
String	numeroColegiado	Número de colegiado único
String	nombreCompleto	Nombre completo del usuario
String	correoElectronico	Correo electrónico único
String	dpi	DPI de 13 dígitos único
Date	fechaNacimiento	Fecha de nacimiento
String	password	Contraseña hasheada
String	rol	Rol del usuario (admin o votante)
Date	createdAt	Fecha de creación
Date	updatedAt	Fecha de actualización

realiza

recibe

VOTES		
ObjectId	_id	Identificador único
ObjectId	usuarioId	Referencia a User
ObjectId	campaniaId	Referencia a Campaign
ObjectId	candidatoId	Referencia a Candidate
Date	fechaVoto	Fecha y hora del voto
Date	createdAt	Fecha de creación
Date	updatedAt	Fecha de actualización